

Optimizing a Natural History Simulation Model of Colorectal Cancer Using GPUs

Carlos Pineda-Antunez¹, David Garibay², Andrew Eckel³, Jorge Roa⁴, Amy B. Knudsen³, Karen M. Kuntz⁵, Nicholson Collier⁶, Jonathan Ozik⁶, Fernando Alarid-Escudero⁷

1 University of Washington, The CHOICE Institute, WA, USA
2 CISIDAT, Cuernavaca, Morelos, Mexico
3 Institute for Technology Assessment, Massachusetts General Hospital, Boston, MA, USA
4 Hertie School, Berlin, Germany
5 University of Minnesota, Minneapolis, MN, USA.
6 Decision and Infrastructure Sciences Division, Argonne National Laboratory, Lemont, IL, United States
7 Stanford University, Stanford, CA, USA.

BACKGROUND:

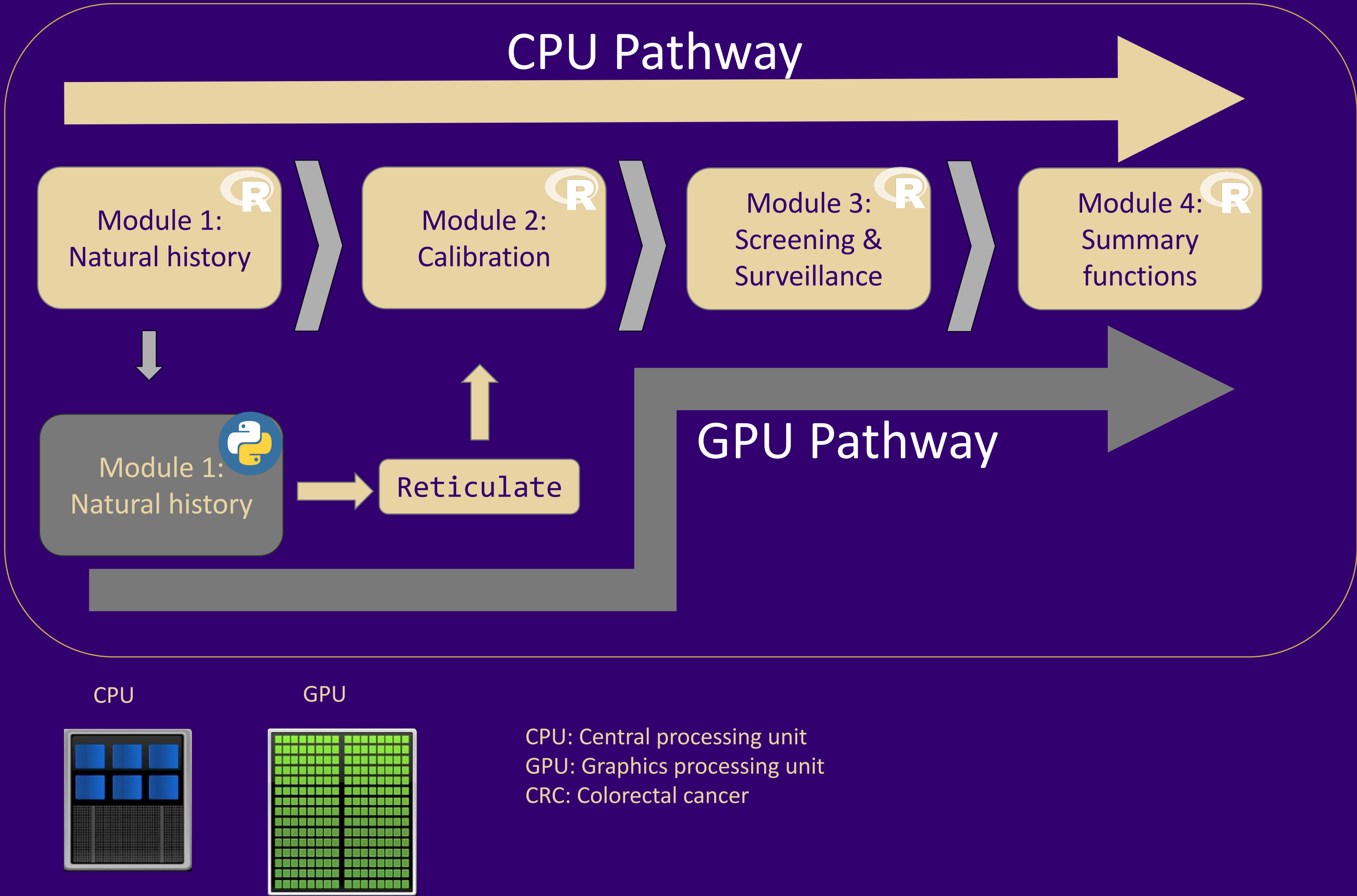
- Microsimulation models are often used to assess the impact of various health policies by simulating disease development, but they tend to be computationally demanding.
- These models are run using CPUs
- GPUs can complete simple and repetitive tasks much faster than CPUs by breaking the tasks down into smaller components.
- GPUs can optimize calculations that can be parallelized.
- The continuous advancement of GPU technology is an opportunity to incorporate it into microsimulation models.
- We aim to optimize the simulation time of microsimulation models using GPUs

METHODS:

- We developed a simplified version of a CISNET colorectal cancer (CRC) simulation model implemented hybridly in R and Python.
- We profiled the code to identify bottlenecks.
- We used Python libraries that support GPU processing to accelerate the most time-consuming modules.
- We compared simulation time between 1 CPU in R, 1 CPU in Python, and 1 GPU in Python. All of them were run on Argonne National Laboratory's Polaris supercomputer cluster.

We **reduced** simulation model execution time by 95% using **GPUs**

CRC microsimulation model



RESULTS:

- The simulation of the natural history and output wrangling were the most computationally time-consuming modules.
- One run of the natural history model for 10 million people took 220 seconds using 1 CPU in R, and went down to 6 seconds when using 1 GPU in Python.
- The accelerated module was incorporated into the R environment using the package reticulate.

Python in CPU	Python in GPU
import pandas as pd	import cudf as pd
import numpy as np	import cupy as np

Table 1. Example of packages importing GPU-suited libraries in Python

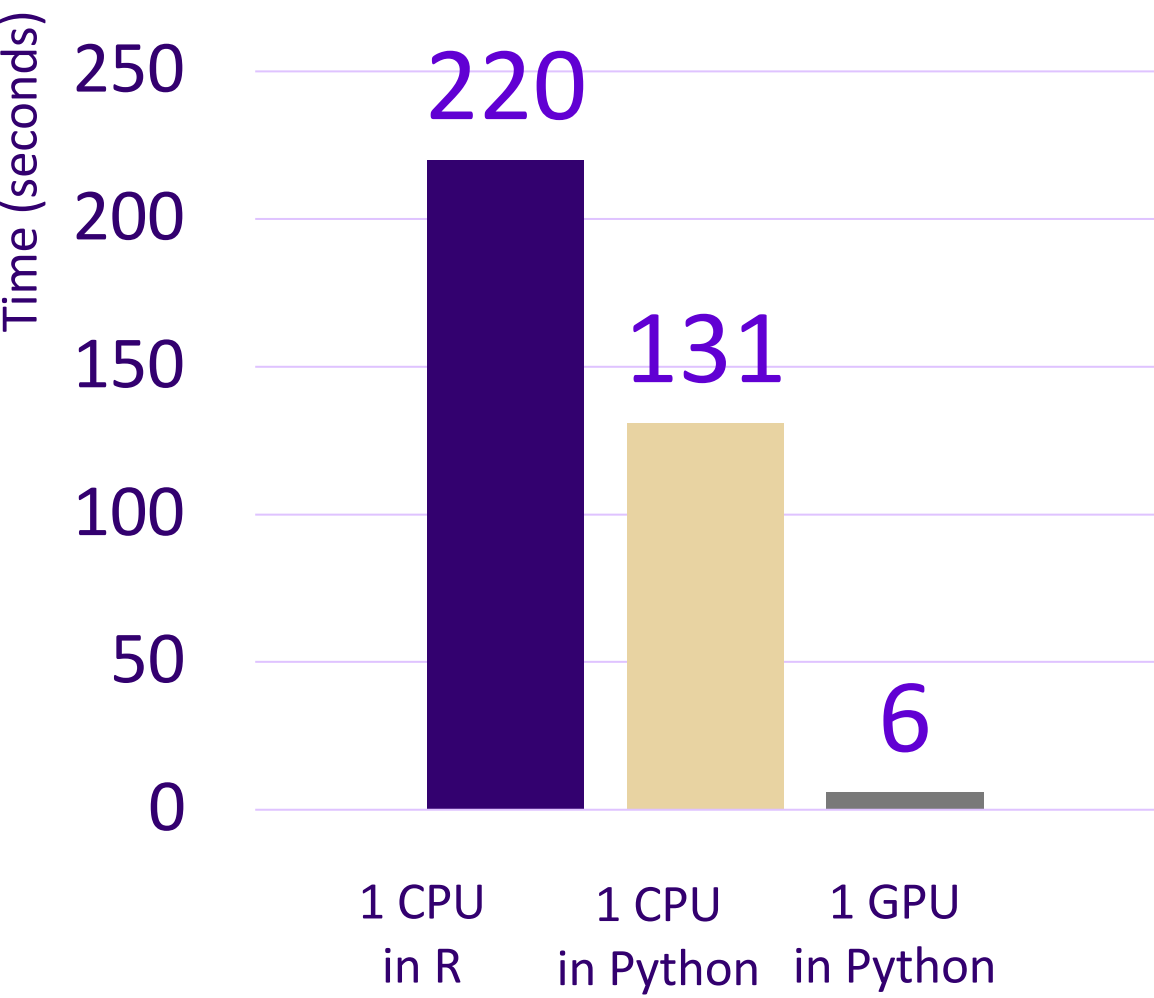


Figure 1. Code performance comparison

DISCUSSION:

We demonstrated how GPU processing can significantly increase microsimulation models' computational efficiency. GPUs greatly reduce the computational burden and time when conducting model calibration, uncertainty quantification, and probabilistic sensitivity analysis.

The grant U01-CA253913 from the National Cancer Institute (NCI) provided financial support for this study.



The content is solely the authors' responsibility and does not necessarily represent the official views of the affiliations represented herein.
Contact e-mail: cpinedaa@uw.edu